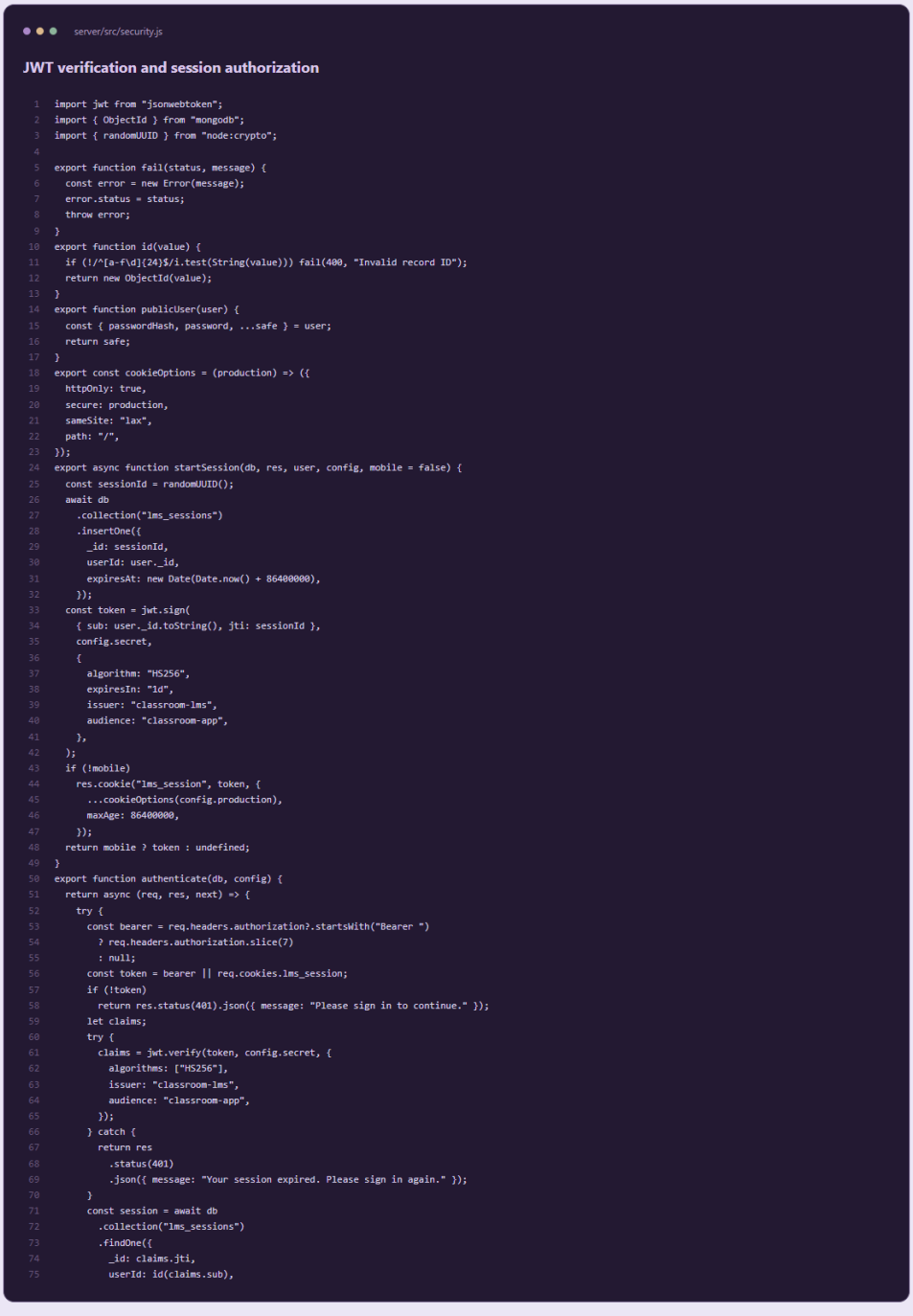
# Source Code Screenshots

Selected implementation excerpts demonstrating authentication, validation, authorization, file handling, React hooks/routing, and Flutter API/submission behavior.

The complete editable source accompanies these screenshots. Secrets and environment configuration are excluded.

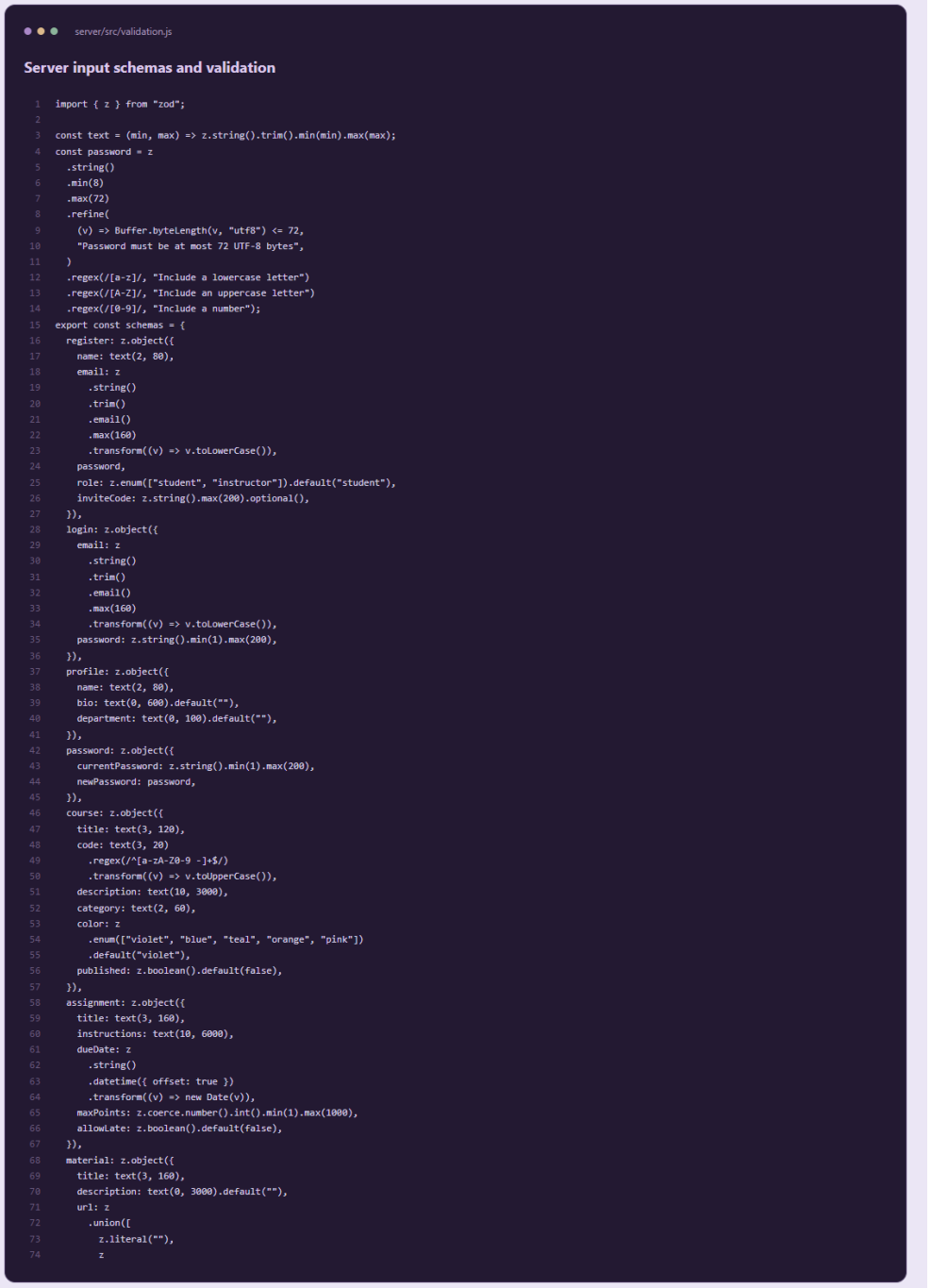
# 1. JWT verification and session authorization

server/src/security.js



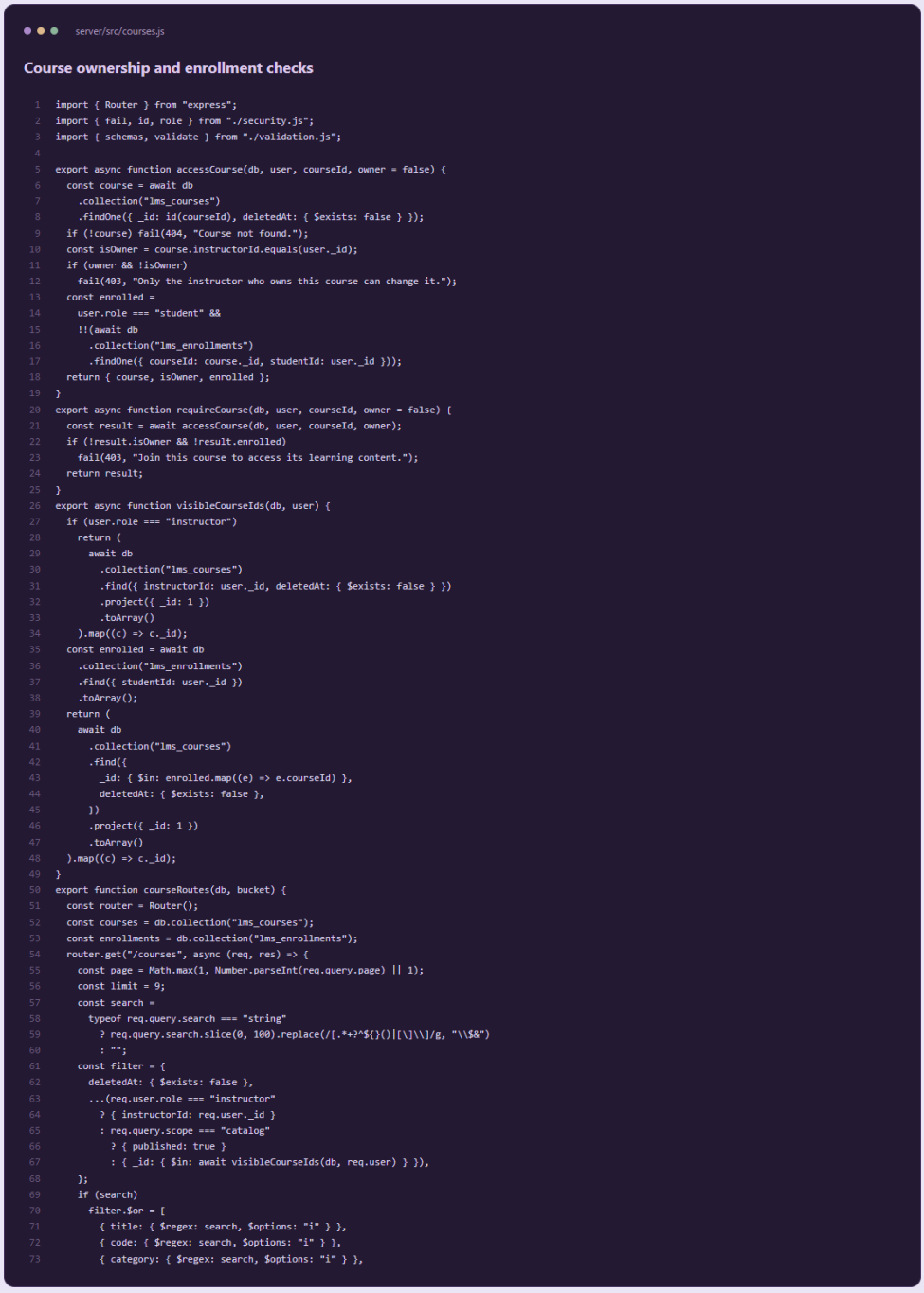
# 2. Server input schemas and validation

server/src/validation.js



# 3. Course ownership and enrollment checks

server/src/courses.js



# 4. File validation and durable upload

server/src/content.js

server/src/content.js

**File validation and durable upload**

```
import { Router } from "express";
import multer from "multer";
import path from "node:path";
import { fail, id, role } from "./security.js";
import { schemas, validate } from "./validation.js";
import { requireCourse } from "./courses.js";

const upload = multer({
  storage: multer.memoryStorage(),
  limits: { fileSize: 20 * 1024 * 1024, files: 1, fields: 6 },
  fileFilter(req, file, next) {
    const allowed = [
      ".pdf",
      ".txt",
      ".csv",
      ".docx",
      ".pptx",
      ".xlsx",
      ".zip",
      ".png",
      ".jpg",
      ".jpeg",
      ".mp4",
    ];
    if (!allowed.includes(path.extname(file.originalname).toLowerCase()))
      return next(
        Object.assign(
          new Error("Use PDF, TXT, CSV, Office, ZIP, PNG, JPG, or MP4 files."),
          { status: 400 },
        ),
      );
    next(null, true);
  },
}).single("file");
function checkFile(file) {
  const ext = path.extname(file.originalname).toLowerCase();
  const b = file.buffer;
  const valid = [".txt", ".csv"].includes(ext)
    ? !b.includes(0)
    : ext === ".pdf"
      ? b.subarray(0, 5).toString() === "%PDF-"
      : [".docx", ".pptx", ".xlsx", ".zip"].includes(ext)
        ? b[0] === 0x50 && b[1] === 0x4b
        : ext === ".png"
          ? b
              .subarray(0, 8)
              .equals(Buffer.from([137, 80, 78, 71, 13, 10, 26, 10]))
          : [".jpg", ".jpeg"].includes(ext)
            ? b[0] === 255 && b[1] === 216 && b[2] === 255
            : b.subarray(4, 8).toString() === "ftyp";
  if (!b.length || !valid)
    fail(
      400,
      "The file contents do not match its extension, or the file is empty.",
    );
}
async function saveFile(bucket, file, user) {
  if (!file) return null;
  checkFile(file);
  const filename = path
    .basename(file.originalname)
    .replace(/[^a-zA-Z0-9._ -]/g, "_")
    .slice(0, 160);
  const stream = bucket.openUploadStream(filename, {
    metadata: { ownerId: user._id, mimeType: file.mimetype },
  });
  await new Promise((resolve, reject) => {
    stream.on("finish", resolve).on("error", reject);
    stream.end(file.buffer);
  });
  return { id: stream.id, name: filename, size: file.size };
}
async function removeFile(bucket, file) {
  if (file?.id)
    await bucket.delete(file.id).catch((error) => {
      if (!/File not found/.test(error.message)) throw error;
    });
}

export function contentRoutes(db, bucket) {
  const router = Router();
  for (const [type, schema] of [
    ["assignments", schemas.assignment],
    ["announcements", schemas.announcement],
  ]) {
    const collection = db.collection(`lms_${type}`);
    router.post(
      `/courses/:courseId/${type}`,
      role("instructor"),
      async (req, res) => {
```

# 5. Protected React routing

client/src/App.jsx

client/src/App.jsx

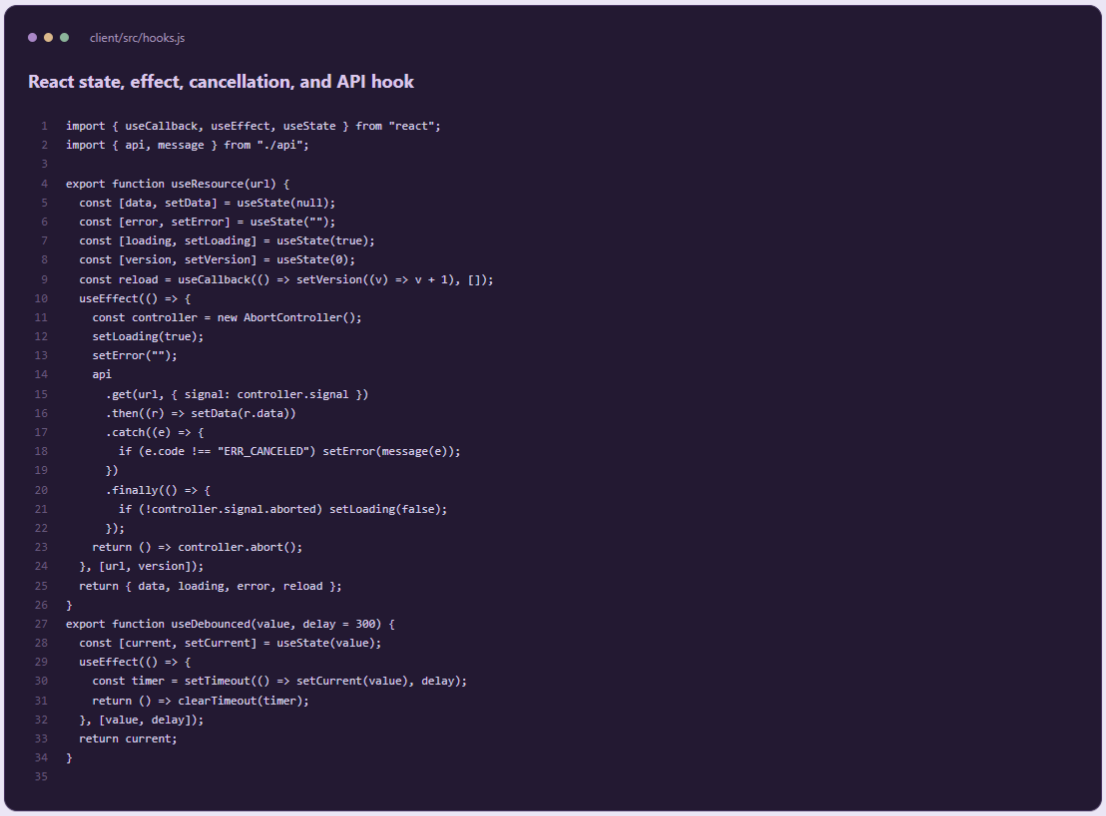
**Protected React routing**

```jsx
import { BrowserRouter, Navigate, Route, Routes } from "react-router-dom";
import { Toaster } from "react-hot-toast";
import { AuthProvider, useAuth } from "./auth";
import AuthPage from "./AuthPage";
import Shell from "./Shell";
import Dashboard from "./Dashboard";
import Courses from "./Courses";
import CourseDetail from "./CourseDetail";
import ActivityPage from "./ActivityPage";
import Profile from "./Profile";
import { ErrorState, Loading } from "./ui";
function Protected() {
  const { user, loading, unavailable, restore } = useAuth();
  if (loading) return <Loading />;
  if (unavailable)
    return (
      <ErrorState
        error="The server is unavailable. Start the API and check your database connection."
        retry={restore}
      />
    );
  return user ? <Shell /> : <Navigate to="/login" replace />;
}
export default function App() {
  return (
    <AuthProvider>
      <BrowserRouter>
        <Routes>
          <Route path="/login" element={<AuthPage />} />
          <Route path="/register" element={<AuthPage register />} />
          <Route path="/signup" element={<Navigate to="/register" replace />} />
          <Route element={<Protected />}>
            <Route index element={<Dashboard />} />
            <Route path="/courses" element={<Courses />} />
            <Route path="/courses/:courseId" element={<CourseDetail />} />
            <Route
              path="/assignments"
              element={<ActivityPage type="assignments" />}
            />
            <Route path="/grades" element={<ActivityPage type="grades" />} />
            <Route
              path="/announcements"
              element={<ActivityPage type="announcements" />}
            />
            <Route path="/profile" element={<Profile />} />
            <Route path="*" element={<Navigate to="/" replace />} />
          </Route>
        </Routes>
      </BrowserRouter>
      <Toaster
        position="top-right"
        toastOptions={{
          duration: 4000,
          style: { fontSize: 14, borderRadius: 12 },
        }}
      />
    </AuthProvider>
  );
}

```

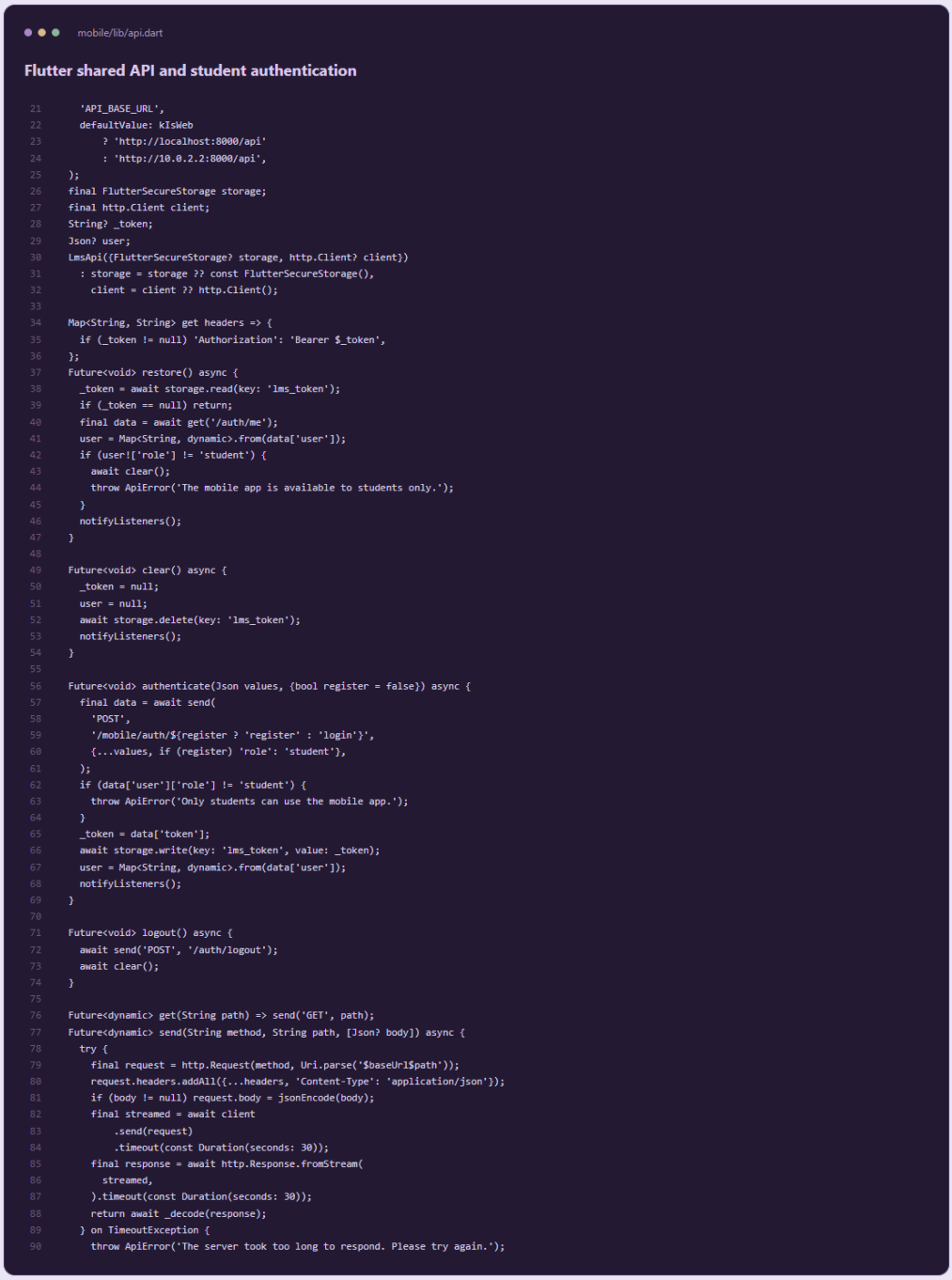
# 6. React state, effect, cancellation, and API hook

client/src/hooks.js



# 7. Flutter shared API and student authentication

mobile/lib/api.dart



# 8. Flutter assignment submission and confirmation

mobile/lib/submission_screen.dart

